Competition Math Question Bank — Project Overview

Project Planning Document

2026-03-07

Contents

1 Executive Summary **1**

2 Core Features **1**
- 2.1 Problem Management . . . 1
  - 2.1.1 Adding Problems . . . 1
  - 2.1.2 Embedded Problems . . . 2
  - 2.1.3 Solutions . . . 3
- 2.2 Tag Taxonomy . . . 3
  - 2.2.1 Primary Topics . . . 4
  - 2.2.2 Example Subtopics . . . 4
  - 2.2.3 Custom Tags . . . 4
- 2.3 Search & Discovery . . . 5
  - 2.3.1 Search Capabilities . . . 5
  - 2.3.2 Advanced Search Features . . . 6
- 2.4 User System . . . 6
  - 2.4.1 Authentication . . . 6
  - 2.4.2 User Roles . . . 7
  - 2.4.3 User Profiles . . . 7

3 Tech Stack Recommendation **7**
- 3.1 Overview . . . 7
- 3.2 Frontend . . . 7
  - 3.2.1 Framework: Next.js (App Router) . . . 7
  - 3.2.2 Styling: Tailwind CSS + shadcn/ui . . . 8
  - 3.2.3 LaTeX Rendering: KaTeX . . . 8
  - 3.2.4 Rich Text / Problem Editor: Tiptap or MDXEditor . 9

3.3 Backend . . . 9
3.3.1 Framework: Next.js API Routes + tRPC . . . 9
3.3.2 ORM: Drizzle ORM . . . 10
3.4 Database . . . 10
3.4.1 Primary Database: PostgreSQL . . . 10
3.4.2 Full-Text Search . . . 11
3.4.3 LaTeX-Aware Search Index . . . 12
3.5 Authentication: NextAuth.js (Auth.js) . . . 12
3.6 File Storage: S3-Compatible Storage . . . 12
3.7 Deployment & Infrastructure . . . 13
3.7.1 Hosting: Vercel . . . 13
3.7.2 CI/CD: GitHub Actions . . . 13
3.7.3 Monitoring: Sentry + Vercel Analytics . . . 13
3.8 Tech Stack Summary Table . . . 14

4 Data Model **14**
4.1 Entity Relationship Overview . . . 14
4.2 Additional Tables . . . 15

5 Branding **15**
5.1 Name Candidates . . . 15
5.1.1 Conceptual / Abstract Names . . . 16
5.1.2 Competitive / Action-Oriented Names . . . 16
5.1.3 Community / Discovery-Oriented Names . . . 16
5.1.4 Recommended Name: **Lemma** . . . 16
5.2 Visual Identity . . . 17
5.2.1 Color Palette . . . 17
5.2.2 Typography . . . 17
5.2.3 Logo Concept . . . 18
5.2.4 Design Principles . . . 18
5.3 Tone of Voice . . . 19

6 Development Roadmap **19**
6.1 Phase 1: MVP (Weeks 1–6) . . . 19
6.2 Phase 2: Community Features (Weeks 7–10) . . . 20
6.3 Phase 3: Scale & Polish (Weeks 11–14) . . . 20
6.4 Phase 4: Advanced Features (Weeks 15+) . . . 21

7 Competitive Landscape **22**

8 Risk Assessment **22**

9 Appendix A: Inspirations & References **22**

10 Appendix B: LaTeX Rendering Examples **23**

1 Executive Summary

This document outlines the architecture, features, tech stack, and branding strategy for a community-driven competition math question bank website. The
platform allows users to contribute, tag, search, and solve competition-level math problems across topics like Geometry, Number Theory, Algebra, and Combinatorics. It supports embedded problems from external sources, user-submitted
solutions, fuzzy search, and difficulty-based filtering.

2 Core Features

2.1 Problem Management

2.1.1 Adding Problems

- Users can submit problems through a rich-text editor that supports LaTeX
  rendering (essential for mathematical notation).

- Each problem submission requires:

  - **Problem statement** — written in LaTeX-compatible markup or pasted as
    plain text with inline LaTeX delimiters (e.g., `$...$` for inline, `$$...$$` for display math).

  - **Tags** — one or more topic tags selected from a predefined taxonomy
    (see Tag Taxonomy below).

- **Difficulty rating** — a numerical scale from 1 to 10, manually assigned
  by the problem provider. This maps loosely to competition levels:

Rating	Approximate Level
1-2	AMC 8 / Introductory
3-4	AMC 10 / Intermediate
5-6	AMC 12 / AIME Early
7-8	AIME Late / USAMO Early
9-10	USAMO / IMO / Research-Level

- **Source attribution** — optional field for competition name, year, and
  problem number (e.g., "IMO 2023, Problem 3").

- **Embed URL** — optional. If the problem originates on another website
  (e.g., AoPS, Mathlinks, Codeforces), the user can paste a URL that
  will be rendered as an iframe embed or a smart link card.

2.1.2 Embedded Problems

- Support for embedding problems from external websites via:

  - **oEmbed protocol** — auto-detect and render rich previews from supported
    sites (Art of Problem Solving, Brilliant, etc.).

  - **iframe embedding** — for sites that allow iframe inclusion, render the
    external problem directly within the platform.

  - **Fallback link card** — if embedding is not possible (due to CSP or
    X-Frame-Options restrictions), display a styled link card with the problem title, source domain, and a "View on [source]" button.

- Embedded problems still carry local metadata (tags, difficulty, solutions)
  so they are fully searchable within the platform.

2.1.3 Solutions

- Each problem can have multiple user-submitted solutions.

- Solutions support full LaTeX rendering, diagrams (uploaded images or TikZ-generated SVGs), and step-by-step formatting.

- Solutions can be upvoted/downvoted by the community, with the highest-rated solution displayed first.

- Solution authors receive attribution and reputation points.

- A "spoiler toggle" hides solutions by default so users can attempt problems before viewing answers.

2.2 Tag Taxonomy

The tagging system uses a two-level hierarchy: **primary topics** and **subtopics**. Users must select at least one primary topic; subtopics are optional but encouraged.

2.2.1 Primary Topics

- Algebra

- Geometry

- Number Theory

- Combinatorics
- Analysis (for olympiad-adjacent or advanced problems)
- Probability & Statistics
- Logic & Proofs

2.2.2 Example Subtopics

Primary Topic	Subtopics
Algebra	Polynomials, Inequalities, Functional Equations, Sequences
Geometry	Euclidean, Projective, Trigonometric, Coordinate, 3D
Number Theory	Divisibility, Modular Arithmetic, Primes, Diophantine Eqns
Combinatorics	Counting, Graph Theory, Pigeonhole, Generating Functions
Probability	Discrete, Continuous, Expected Value, Combinatorial Prob.

2.2.3 Custom Tags

- Users may propose new subtopics. Proposed tags enter a moderation queue
  and are approved by admins or through community consensus (e.g., 5+ users
  have used the same proposed tag).

2.3 Search & Discovery

2.3.1 Search Capabilities

The search system is a critical differentiator and must be fast, flexible, and math-aware. It supports three primary search axes:

1. 1. Topic-Based Search (Tag Filtering)
   - Filter by one or more primary topics and subtopics.

- Boolean logic: users can combine tags with AND/OR operators.
  - Example: `Geometry AND Inequalities` returns problems tagged with both.
  - Example: `Number Theory OR Combinatorics` returns problems tagged with either.
- Tag chips are displayed as interactive filter pills in the UI.

2. 2. Full-Text Fuzzy Search
   - Search over problem statements, solution text, and source attributions.
   - Uses fuzzy matching to tolerate typos and partial matches.
     - Example: Searching "triangel incircle" should match problems containing
       "triangle" and "incircle."
   - Must handle LaTeX-aware searching:
     - Strip LaTeX commands for indexing so that searching "integral" matches
       problems containing `\int`.
     - Index both the raw LaTeX and a "plain text" rendering of the math.
   - Search results are ranked by relevance (BM25 or similar) with recency
     and vote count as secondary signals.
3. 3. Difficulty-Based Filtering
   - Slider or dropdown to filter by difficulty range (e.g., 3–6).

- Can be combined with topic filters and text search.

2.3.2 Advanced Search Features

- **Sort options**: by relevance, difficulty, date added, number of solutions,
  popularity (views/votes).

- **Saved searches**: logged-in users can save search queries for quick access.

- **"Similar problems" recommendations**: after viewing a problem, the system
  suggests related problems based on tag overlap and content similarity (using vector embeddings for semantic search as a stretch goal).

2.4 User System

2.4.1 Authentication

- Email/password registration with email verification.

- OAuth login via Google and GitHub (common in math/CS communities).

- Optional: Discord OAuth (many competition math communities use Discord).

2.4.2 User Roles

Role	Permissions
Viewer	Browse, search, and view problems/solutions (no account needed)
Contributor	Submit problems, submit solutions, vote, comment
Moderator	Edit/delete any problem or solution, approve tags, manage flags
Admin	Full access including user management and site configuration

2.4.3 User Profiles

- Display name, avatar, and bio.
- Stats: problems submitted, solutions submitted, total upvotes received.
- Activity history: list of contributed problems and solutions.
- Bookmarked problems (private list for personal study).

3 Tech Stack Recommendation

3.1 Overview

The recommended stack prioritizes developer experience, performance, strong TypeScript support, and excellent LaTeX rendering. The architecture follows a modern full-stack JavaScript/TypeScript approach.

3.2 Frontend

3.2.1 Framework: Next.js (App Router)

- **Why Next.js**: It provides server-side rendering (SSR) and static site generation (SSG), which are excellent for SEO (important for a public question bank that should be discoverable via Google). The App Router
(introduced in Next.js 13+) offers React Server Components, nested layouts, and streaming — all useful for a content-heavy site.
- **Alternative considered**: Remix (also excellent for SSR, but Next.js has a larger ecosystem and more community resources).

3.2.2 Styling: Tailwind CSS + shadcn/ui

- **Tailwind CSS**: Utility-first CSS framework that enables rapid, consistent styling without writing custom CSS files. Pairs perfectly

with component-based architectures.

- **shadcn/ui**: A collection of beautifully designed, accessible, and customizable components built on Radix UI primitives. Provides ready-made components for dialogs, dropdowns, command palettes, and form elements — saving significant development time.

- **Why not Material UI or Chakra**: shadcn/ui components are copied into
  your codebase (not installed as a dependency), giving you full control over styling and behavior. This avoids the "fighting the framework" problem common with opinionated component libraries.

3.2.3 LaTeX Rendering: KaTeX

- **KaTeX** is the recommended LaTeX renderer because it is significantly faster than MathJax (renders synchronously, no reflow) and produces high-quality output.

- Integrate via `react-katex` or `rehype-katex` (if using MDX for problem content).

- For the problem editor, use a split-pane editor with live KaTeX preview (similar to Overleaf's interface).

3.2.4 Rich Text / Problem Editor: Tiptap or MDXEditor

- **Tiptap**: A headless, extensible rich-text editor built on ProseMirror. Supports custom extensions for LaTeX blocks, code blocks, and image uploads. Highly customizable.

- **Alternative**: A Markdown/MDX editor with a custom LaTeX plugin, using
  `@mdxeditor/editor` for a more lightweight approach.

- The editor should support:
  - Inline and display LaTeX with live preview.
  - Image upload (for geometric diagrams).
  - Code blocks (for computational problems).
  - Ordered/unordered lists for multi-part problems.

3.3 Backend

3.3.1 Framework: Next.js API Routes + tRPC

- **Next.js API Routes**: Since the frontend is already Next.js, using its built-in API routes avoids the overhead of a separate backend server. For a project of this scale, this is sufficient.

- **tRPC**: Provides end-to-end type safety between frontend and backend without code generation. Define your API procedures in TypeScript, and
  the frontend client automatically knows the input/output types. Dramatically reduces bugs and improves developer velocity.

- **Alternative considered**: A separate Express/Fastify backend. This is
  overkill unless you anticipate the backend needing to serve non-web clients (e.g., a mobile app) in the near future.

3.3.2 ORM: Drizzle ORM

- **Drizzle** is a lightweight, type-safe ORM for TypeScript that generates efficient SQL. It is less "magical" than Prisma (no separate schema language, no heavy client generation) and gives you more control over queries.

- **Why not Prisma**: Prisma is excellent but generates a heavy client, has a separate schema language (.prisma), and can be slower for complex
queries. Drizzle's SQL-like query builder is more transparent.

- **Alternative**: Prisma is still a strong choice if the team prefers its developer experience and migration tooling.

3.4 Database

3.4.1 Primary Database: PostgreSQL

- PostgreSQL is the gold standard for relational databases. It handles complex queries, full-text search, JSONB fields (useful for flexible problem metadata), and scales well.

- **Hosting options**:

  - **Neon** — serverless Postgres with branching (great for development).

  - **Supabase** — managed Postgres with built-in auth, storage, and real-time subscriptions.

  - **Railway** or **Render** — simple managed Postgres.

  - **Self-hosted** — on a VPS if cost is a primary concern.

3.4.2 Full-Text Search

1. Option A: PostgreSQL Native FTS (Recommended for MVP)

   - PostgreSQL has built-in full-text search with `tsvector` and `tsquery`.

   - Supports ranking (`ts_rank`), stemming, and language-aware tokenization.

- For fuzzy matching, combine with `pg_trgm` (trigram matching) which
  enables `%` similarity operator and fuzzy `LIKE` queries.

- **Pros**: No additional infrastructure, good enough for tens of thousands
  of problems, well-integrated with your existing database.

- **Cons**: Not as feature-rich as dedicated search engines for very large
  datasets.

2. Option B: Meilisearch (Recommended for Scale)

   - If the dataset grows beyond ~100k problems or if you need more sophisticated relevance tuning, deploy Meilisearch as a dedicated search backend.

   - Meilisearch provides typo-tolerance, faceted filtering (by tag, difficulty), and sub-50ms response times out of the box.

   - It is simpler to operate than Elasticsearch and has excellent documentation.

   - **Alternative**: Typesense (similar to Meilisearch, slightly different tradeoffs in relevance tuning).

3.4.3 LaTeX-Aware Search Index

- Regardless of which search engine you use, build a preprocessing pipeline that:

  1. Strips LaTeX commands (`\frac{a}{b}` → "a over b" or "a/b").

  2. Normalizes mathematical symbols (`\geq` → "" and "greater than or equal").

3. Indexes both the raw LaTeX and the normalized plain-text version.

- This allows users to search for "triangle inequality" and match problems that contain `$\triangle ABC$` and "inequality."

3.5 Authentication: NextAuth.js (Auth.js)

- Supports email/password, Google OAuth, GitHub OAuth, and Discord OAuth.

- Integrates seamlessly with Next.js and can store sessions in the database (via the Drizzle adapter).

- Handles JWT or database sessions, CSRF protection, and secure cookies
  out of the box.

3.6 File Storage: S3-Compatible Storage

- For user-uploaded images (diagrams, handwritten solutions):

  - **Cloudflare R2** — S3-compatible, no egress fees, generous free tier.

  - **AWS S3** — industry standard, slightly higher cost.

  - **Uploadthing** — a developer-friendly wrapper around S3 designed for
    Next.js applications.

3.7 Deployment & Infrastructure

3.7.1 Hosting: Vercel

- **Vercel** is the natural deployment target for Next.js applications.
  It provides:

  - Automatic preview deployments for every pull request.
  - Edge caching and CDN for static assets.
  - Serverless functions for API routes.
  - Built-in analytics and monitoring.

- **Alternative**: Self-host on a VPS (e.g., Hetzner, DigitalOcean) using Docker if you want to avoid vendor lock-in or reduce costs at scale.

3.7.2 CI/CD: GitHub Actions

- Automated testing, linting, and deployment on push/PR.

3.7.3 Monitoring: Sentry + Vercel Analytics

- Sentry for error tracking and performance monitoring.
- Vercel Analytics for traffic and Core Web Vitals.

3.8 Tech Stack Summary Table

Layer	Technology	Purpose
Frontend	Next.js (App Router)	SSR/SSG React framework
Styling	Tailwind CSS + shadcn/ui	Utility-first CSS + components
LaTeX	KaTeX	Fast math rendering
Editor	Tiptap	Rich-text problem/solution input
API	tRPC	Type-safe API layer
ORM	Drizzle ORM	Type-safe database queries
Database	PostgreSQL	Primary data store
Search	pg_{trgm} + tsvector (MVP)	Full-text + fuzzy search
Search (scale)	Meilisearch	Dedicated search engine
Auth	NextAuth.js (Auth.js)	Authentication & sessions
Storage	Cloudflare R2	Image/file uploads
Hosting	Vercel	Deployment & CDN
Monitoring	Sentry	Error tracking

4 Data Model

4.1 Entity Relationship Overview

```
   User      1:N    Problem        N:M     Tag

id                   id                      id
name                 title                   name
email                statement (LaTeX)       slug
avatar_url           difficulty (1-10)       parent_id
role                 source                  description
created_at           embed_url
       author_id (FK)
                      created_at
                      updated_at
                      view_count
                      search_vector



      1:N    Solution

                       id
                       body (LaTeX)
```

```
problem_id (FK)
author_id (FK)
upvotes
downvotes
created_at
```

4.2 Additional Tables

- `problem_tags` — join table for the N:M relationship between problems and tags.

- `votes` — tracks which user voted on which solution (prevents double-voting).

- `bookmarks` — tracks which user bookmarked which problem.

- `comments` — optional, for discussion threads on problems or solutions.

5 Branding

5.1 Name Candidates

The name should evoke mathematical rigor, competition, and community. Here are
several directions:

5.1.1 Conceptual / Abstract Names

Name	Rationale
Lemma	A proven statement used to prove a theorem — suggests building blocks. Short, memorable, mathematical.
Conjecture	An unproven statement — evokes challenge and discovery.
Axiom	A foundational truth — suggests reliability and fundamentals.
Proof Garden	Problems are "cultivated" — suggests growth and community.
The Archive	Simple, authoritative, suggests a vast repository.

5.1.2 Competitive / Action-Oriented Names

Name	Rationale
Olympiad Vault	Directly targets competition math. "Vault" = treasure trove.
ProofArena	Competition-forward, suggests a place to test skills.
MathForge	Problems are "forged" — suggests craftsmanship and creation.
QED Bank	QED (quod erat demonstrandum) is iconic in math proofs.

5.1.3 Community / Discovery-Oriented Names

Name	Rationale
Mathonauts	"Explorers of mathematics" — playful, community feel.
Problem Atelier	"Atelier" = workshop — suggests craft and collaboration.
Sigma Library	Sigma () is a universal math symbol — authoritative feel.

5.1.4 Recommended Name: Lemma

- Two syllables, easy to say and spell.
- Instantly recognizable to anyone with a math background.
- Works as a domain: `lemma.math`, `getlemma.io`, `lemma.so`.
- Extensible: "Lemma" can grow from a question bank to a full learning platform without the name feeling limiting.

5.2 Visual Identity

5.2.1 Color Palette

The palette should feel intellectual, clean, and energetic without being childish. Competition math communities tend to prefer dark-mode-friendly designs.

Role	Color	Hex	Usage
Primary	Deep Indigo	`#3730A3`	Headers, primary buttons, links
Secondary	Warm Amber	`#D97706`	Accents, badges, difficulty labels
Background	Slate White	`#F8FAFC`	Light mode background
Dark BG	Charcoal	`#0F172A`	Dark mode background
Text	Slate 800	`#1E293B`	Body text (light mode)
Text Dark	Slate 200	`#E2E8F0`	Body text (dark mode)
Success	Emerald	`#059669`	Correct indicators, upvotes
Danger	Rose	`#E11D48`	Errors, downvotes, warnings
Tag Geometry	Teal	`#0D9488`	Geometry tag color
Tag Number	Purple	`#7C3AED`	Number Theory tag color
Tag Algebra	Blue	`#2563EB`	Algebra tag color
Tag Combo	Orange	`#EA580C`	Combinatorics tag color

5.2.2 Typography

- **Headings**: Inter (clean, geometric, excellent readability at all sizes) or Space Grotesk (slightly more distinctive, techy feel).

- **Body text**: Inter or IBM Plex Sans (excellent for long-form reading).

- **Math / Code**: JetBrains Mono or Fira Code (monospace, for code blocks
  and inline code).

- **LaTeX**: KaTeX's default Computer Modern-derived font (users expect this for mathematical content — do not override it).

5.2.3 Logo Concept

- The logo should incorporate the Greek letter (lambda) — visually distinctive and references "Lemma."

- Style: geometric, minimal, possibly constructed from clean lines or a grid. Think of how Stripe, Linear, or Notion approach their logos —

simple geometry that scales from favicon to billboard.

- The lambda could double as a visual metaphor: its two strokes meeting at a point suggest convergence, proof, and solution.

5.2.4 Design Principles

1. **Math-first**: LaTeX rendering should be beautiful and prominent. Problems should look like they belong in a typeset competition paper.

2. **Clean hierarchy**: Clear visual distinction between problem statement,
metadata (tags, difficulty, source), and solutions.

3. **Dark mode native**: Competition math students often study late. Dark mode should be a first-class experience, not an afterthought.

4. **Information density**: Power users (competitive math students) prefer
information-dense layouts over excessive whitespace. Think "Hacker News density" rather than "Medium density."

5. **Accessible**: All colors must meet WCAG AA contrast ratios. Interactive
elements must be keyboard-navigable.

5.3 Tone of Voice

- **Precise**: Mirror the precision of mathematical writing. Avoid unnecessary filler or vague language in UI copy.

- **Encouraging**: "No solution yet — be the first to solve this!" rather than "No solutions found."

- **Community-oriented**: "Contributed by" rather than "Uploaded by." "Join the discussion" rather than "Leave a comment."

- **Understated confidence**: The platform should feel authoritative without
being elitist. Welcome beginners while respecting advanced users.

6 Development Roadmap

6.1 Phase 1: MVP (Weeks 1–6)

- ☐ Project scaffolding (Next.js + Tailwind + shadcn/ui + tRPC + Drizzle)

- ☐ Database schema and migrations (PostgreSQL)

- ☐ User authentication (NextAuth.js with email + Google OAuth)

- ☐ Problem CRUD (create, read, update, delete)

- ☐ LaTeX rendering with KaTeX

- ☐ Basic tag system (predefined tags, single-select)

- ☐ Difficulty rating on problem submission

- ☐ Basic search (PostgreSQL full-text search + pg_{trgm} fuzzy matching)

- ☐ Solution submission with LaTeX support

- ☐ Responsive layout (mobile + desktop)

- ☐ Dark mode

6.2 Phase 2: Community Features (Weeks 7–10)

- ☐ Solution voting (upvote/downvote)
- ☐ User profiles and activity history
- ☐ Bookmarking problems
- ☐ Multi-tag filtering with AND/OR logic
- ☐ Advanced search UI (combined topic + difficulty + text filters)
- ☐ Embed support for external problem URLs
- ☐ Spoiler toggle for solutions
- ☐ Basic moderation tools (flag, edit, delete)

6.3 Phase 3: Scale & Polish (Weeks 11–14)

- ☐ Migrate search to Meilisearch (if dataset warrants it)
- ☐ LaTeX-aware search preprocessing pipeline
- ☐ Image upload for diagrams (Cloudflare R2)
- ☐ Rich text editor (Tiptap) with live LaTeX preview
- ☐ User reputation system

- ☐ Custom tag proposals and moderation queue
- ☐ SEO optimization (structured data, sitemaps, meta tags)
- ☐ Performance optimization (caching, lazy loading)

6.4 Phase 4: Advanced Features (Weeks 15+)

- ☐ "Similar problems" recommendations (vector embeddings)
- ☐ Problem sets / collections (curated lists by topic or competition)
- ☐ Export to PDF (for printing problem sets)
- ☐ API for external integrations
- ☐ Discord bot integration (post daily problems)
- ☐ Contest mode (timed problem sets)
- ☐ Analytics dashboard for contributors

7 Competitive Landscape

Understanding the existing tools helps position this platform:

Platform	Strengths	Gaps This Project Fills
Art of Problem Solving (AoPS)	Massive community, rich history	Dated UI, weak search, no modern tagging or difficulty filtering
Evan Chen's site	High-quality curated problems	Static, no community contributions, no search
KSEA / AMC sites	Official problem archives	No solutions, no tagging, poor UX
Brilliant.org	Beautiful UI, guided learning	Not competition-focused, paywalled
This Project (Lemma)	Modern UI, fuzzy search, community solutions, embeds, open tagging	New, needs to build community and content critical mass

8 Risk Assessment

Risk	Likelihood	Impact	Mitigation
Low initial content volume	High	High	Seed with 500+ problems from public competition archives (AMC, AIME, IMO).
LaTeX rendering bugs	Medium	Medium	Use battle-tested KaTeX; extensive QA on edge cases.
Search relevance issues	Medium	High	Start with pg_{trgm} (simple, debuggable); iterate based on user feedback.
Embed CSP/CORS failures	High	Low	Graceful fallback to link cards; maintain whitelist of embeddable sources.
Spam / low-quality content	Medium	Medium	Reputation system, moderation queue, rate limiting on submissions.
Scaling database search	Low (MVP)	High	Planned migration path to Meilisearch.

9 Appendix A: Inspirations & References

- **Art of Problem Solving** (aops.com) — the incumbent in competition math.

- **Codeforces** (codeforces.com) — excellent problem tagging and difficulty rating system.

- **LeetCode** (leetcode.com) — strong search, filtering, and community solution model.

- **Notion** — clean, minimal UI that balances information density with aesthetics.

- **Linear** — exemplary dark mode design and keyboard-first UX.

- **Evan Chen's Handouts** (web.evanchen.cc) — gold standard for competition math content quality.

10 Appendix B: LaTeX Rendering Examples

For reference, these are the kinds of expressions the platform must render correctly:

```
Inline: Let $p$ be a prime and $a \not\equiv 0 \pmod{p}$.

Display:
$$\sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k} = (x+y)^n$$

Aligned:
\begin{align*}
  a^2 + b^2 &= c^2 \\
  e^{i\pi} + 1 &= 0
\end{align*}

Geometry:
In $\triangle ABC$, let $O$ be the circumcenter and $H$ the orthocenter.
Prove that $\vec{OH} = \vec{OA} + \vec{OB} + \vec{OC}$.
```

The editor and renderer must handle all standard LaTeX math packages including `amsmath`, `amssymb`, and `amsthm` environments.